

AgenTekki

Steuererklärung 2026, Kanton Zürich

A tax return filed end to end by an AI agent

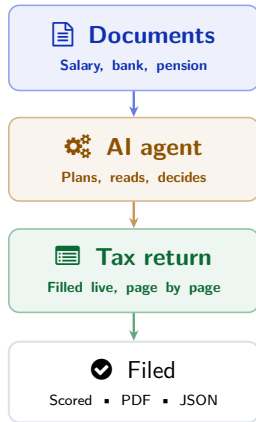
What AgenTekki is

The problem

A Zurich return spans **28 pages**. The figures are not in the form — they are buried in documents, and the rules that combine them live in cantonal guidance.

What we built

An **AI agent** that reads the documents, looks up the rules it needs, and fills the whole return **without human input** — while you watch it happen.



Two halves of the same thing

The exercise — what students build

A Colab notebook. Students write the agent's reasoning: the planning prompt, the retrieval step, the execute step.

They never touch a frontend or a backend.

💡 `think_step()`



The demo — what it becomes

The same agent, wired into a real product: a browser UI, a server, live updates, a score and a filed PDF.

This is what you are about to see.

💻 The application

The demo reads the **same personas, guides, MCP server and scorer** straight from the course repository — not copies. What it shows can never drift from what students are graded on.

How the pieces talk



⇔ 1 A REST request

The browser asks the server to do something and waits for one reply.
POST /api/sessions

POST .../run
GET /api/personas
GET .../score

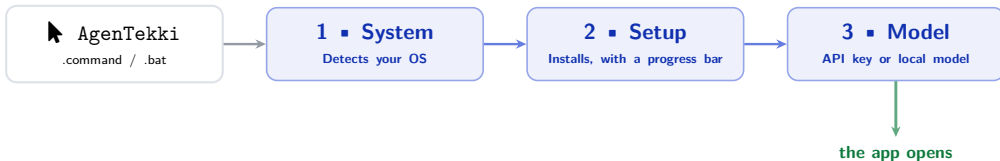
⚡ 2 SSE, for live updates

A run takes minutes, so the browser cannot just wait. **Server-Sent Events** keeps *one* connection open and the server writes into it as things happen — a field filled, a page turned. One-way and plain HTTP, unlike WebSockets. A 1.5s poll runs alongside in case it drops.

🔧 3 And the tools

The agent never touches the DOM. It calls the **nine MCP tools** — scan_page, fill_field, ... Here they are ordinary Python calls inside Flask. The same server *can* speak MCP over stdio to Claude Code or Cursor — same tools, different transport.

Getting it running: double-click



No Python needed

If the machine has none, the launcher fetches a private copy into its own folder. Nothing is installed system-wide.

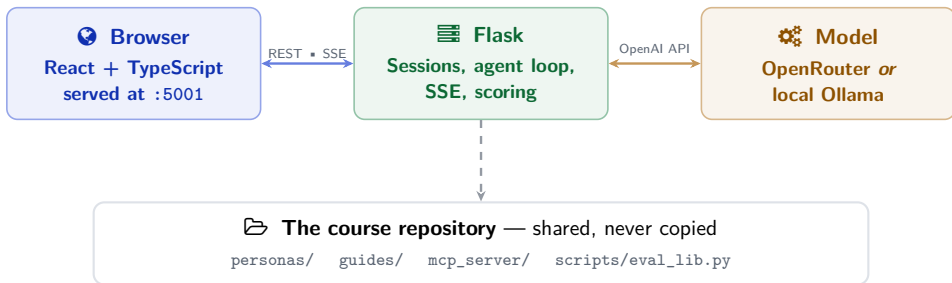
No Node needed

The interface ships pre-built and is served by the backend, so nobody runs `npm`.

Asked once

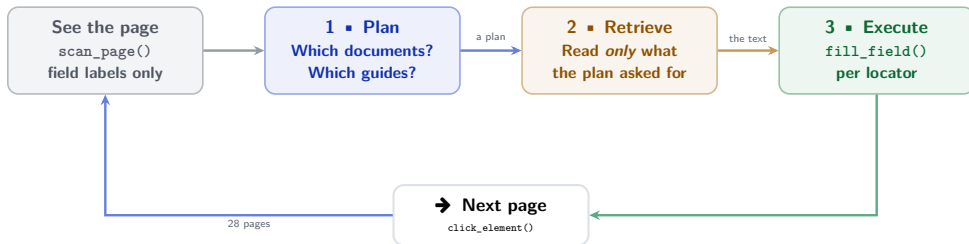
The choice is remembered; later runs go straight to the dashboard.

System architecture



The backend owns no tax knowledge of its own. Documents, rules, the nine tools and the scorer all come from the repository the students work in.

The agent loop: plan, retrieve, execute



Why plan before reading

The planning call sees document **names** only, never their contents. It has to decide what is worth reading — so the call that fills the fields gets a short, relevant context instead of everything at once.

2

model calls
per page

18

pages that
need filling

~50

calls per
return

The nine tools the agent is given

The form

`scan_page()`
every field on this page

`fill_field()`
write one value

`click_element()`
navigate, add rows

`submit_form()`
file the return

The evidence

`list_documents()`
names, never contents

`read_document()`
txt, csv, pdf, json

`list_guides()`
the rule catalogue

`fetch_guide()`
one rule, in full

The taxpayer

`ask_user()`
put a question to the simulated
taxpayer when the documents do
not settle it

Discovery, not guessing

The agent cannot invent a locator.
It has to call `scan_page()` and use
what comes back.

Two ways to power it

Hosted — OpenRouter default

- The **same route the student notebook uses**, so a demo run and a student run are comparable
- A return finishes in a **couple of minutes**
- Needs a key, and the data leaves the machine

Local — Ollama

- **Nothing leaves the laptop**, no key, works offline
- Free, no rate limits
- **15–40 minutes** per return, and less accurate

For **real** tax data local is the responsible choice — financial records should not be posted to a third party. For a **live demo** on fictional taxpayers, hosted wins: the room does not wait half an hour.

Running it locally

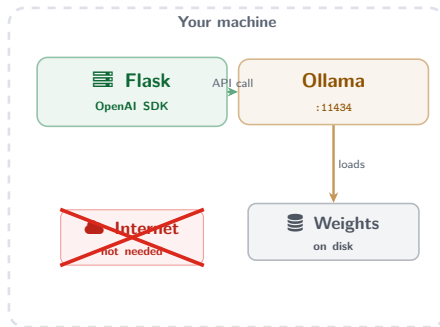
What Ollama is

An open-source runtime that serves models on your own machine behind an **OpenAI-compatible API** — so the same backend code drives local and hosted models unchanged.

The wizard picks a model to suit the machine:

Memory	Suggested	Size
8 GB	gemma3:1b	0.9 GB
16 GB	gemma3:4b	3.3 GB
32 GB+	gemma3:12b	8.1 GB

Models too big for the machine are switched off, not offered.



The taxpayers

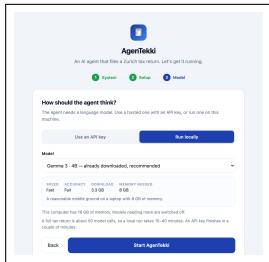
What a persona is

A **fictional taxpayer** with a full document set — salary certificate, bank statement, receipts, pension confirmation — plus a hidden answer key.

The demo ships the **same five** the exercise uses, read straight from the repository. You can also add your own taxpayer through the dashboard; it simply has no answer key to score against.

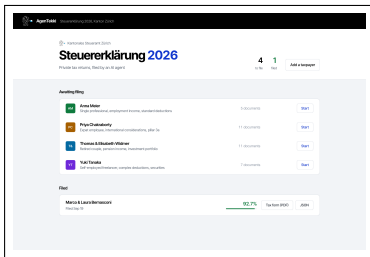
Taxpayer	Situation	Level
Anna Meier	Employed, single	Easy
Priya Chakraborty	Expat, Pillar 3a	Medium
Thomas & Elisabeth Widmer	Retired, investments	Medium
Marco & Laura Bernasconi	Dual income, children, property	Hard
Yuki Tanaka	Freelance, securities	Hard

What it looks like



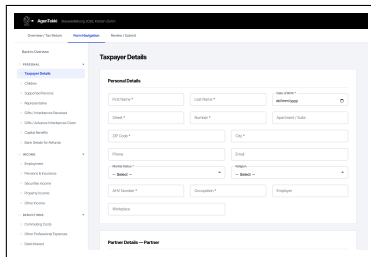
Setup, once

Pick a model; the wizard
does the rest



The register

Who is waiting, who is filed,
and how well



The return

The agent fills these fields
while you watch

One rule, worked through

The Berufsauslagen-Pauschale

Nothing in the documents states this figure. The agent must fetch the rule, find the right line, and do the arithmetic:

- 3% of *Nettolohn* — Lohnausweis field 12
- clamped to CHF 2 000–4 000
- **per person**, summed for a couple

For Anna, field 12 is CHF 125 505.40 — so **CHF 3 765**.

Where agents slip

Take the gross salary instead of field 12 and you get **3 782** — close enough to look right, and wrong.

And a second trap

Her profile says she works at “Google Zurich”; the salary certificate says **Google Switzerland GmbH**. The document wins — a real run got this wrong.

How the run is graded

The same scorer that grades students

Every filled value is compared against the persona's hidden answer key by **type**, never guessed:

- **Numbers** within 1 CHF
- **Choices** through an alias table — ledig counts as single
- **Dates** parsed, not string-matched
- **Free text** fuzzy-matched, then judged by a model if still unclear

Over-filling is penalised

A value the agent invents where the answer key wants nothing costs **half a point**.
Filling everything is not a winning strategy.

88%

a typical
hosted run

25

values in
Anna's key

What comes out



The tax form

Laid out as the canton's
paper return, numbered
Ziffern and all



</> The raw values

Every field the agent filled, as JSON



 The score

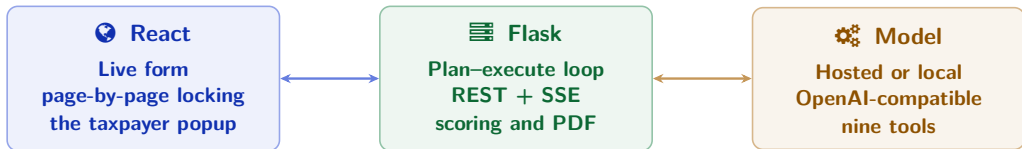
Field by field, with
what was expected

The PDF is generated server-side from the same submission the scorer reads: Swiss thousands separators, income and deduction subtotals, *Reineinkommen* and *Reinvermögen*.

It carries a footer saying it is a simulation on a fictional taxpayer, and stops short of computing tax owed — that needs the cantonal tariff, and a made-up number would be worse than none.

[illegible]

In one picture



An agent that **reads the documents, looks up the rules,**
and **files 28 pages** — then tells you how well it did.

Double-click `AgenTekki.command` ▪ the app opens at `localhost:5001`

Questions?

AgenTekki — a Zurich tax return, filed by an AI agent
